

# Методы оптимизации и исследование операций · лекция 1

## Организационное

- После лекции будет **небольшой тест по материалу** лекции.
  - Тест проходим в **Moodle**: «Сейчас все переходим в Moodle. Я сейчас изменю время, открою тест и проходим его».
  - Тест открывается сразу после лекции; по словам преподавателя, «сегодня только один тест, потому что практики еще не у всех прошли».
  - Завтра — **продолжение практических занятий у второй, первой и третьей групп**.
  - Ранее (со слов преподавателя) по динамическому программированию запланированы «две или три» практики (точное количество преподаватель не уточнила).
- 

## Лекция №6. Динамическое программирование

### Тема

Динамическое программирование: основные понятия, принцип оптимальности Беллмана, постановка задачи, функциональные уравнения, разбор классических задач (кратчайший путь, замена оборудования, распределение ресурсов), отличие от «жадных» алгоритмов.

### План лекции

1. Определение динамического программирования, историческая справка (Беллман, 40-е годы).
  2. Принцип оптимальности (1953).
  3. Постановка задачи, функция Беллмана, функциональные уравнения.
  4. Общая схема решения: условная и безусловная оптимизация.
  5. Свойства задач, решаемых методом динамического программирования.
  6. Примеры многошаговых операций.
  7. Демонстрационный пример: поиск наивыгоднейшего пути.
  8. Задача о замене оборудования.
  9. Задача оптимального распределения ресурсов.
  10. Достоинства и недостатки динамического программирования.
  11. Рекурсия versus динамическое программирование.
  12. «Жадные» алгоритмы (обзорно, на примере алгоритма Дейкстры).
- 

### 1. Определение и история

**Динамическое программирование** — раздел математического программирования, один из разделов оптимального программирования, в котором процесс принятия решения и управления разбивается на несколько этапов (шагов).

- Впервые использовано в **40-х годах XX века** американским математиком **Ричардом Беллманом**.
- Применяется для описания процесса нахождения решения задач, где ответ на одну задачу может быть получен только после решения задачи, предшествующей ей.

- Позволяет одну сложную задачу со многими переменными свести к задаче с малым количеством переменных.

В отличие от линейного программирования (где симплекс-метод является универсальным), в динамическом программировании **универсального метода не существует**. Основной метод — **метод рекуррентных соотношений**.

Процессы принятия решений, построенные по данному принципу, называют **многошаговыми процессами**.

---

## 2. Принцип оптимальности Беллмана

Сформулирован в **1953 году** и лежит в основе динамического программирования.

**Принцип оптимальности:** каково бы ни было состояние системы на каждом этапе, будет приниматься такое решение, которое обеспечивает оптимальность с данного этапа до конца процесса.

То есть на каждом этапе необходимо принимать решение и просматривать его последствия до самого конца процесса.

Если задача, например, о распределении ресурсов на несколько лет деятельности предприятия, то в качестве шага выступает **временной период** (год, день, час) — в зависимости от конкретной задачи.

---

## 3. Постановка задачи динамического программирования

Пусть в начальный момент времени система находится в состоянии  $S_0$ .

- **1-й шаг:** под действием управления  $x_1$  система переходит из состояния  $S_0$  в состояние  $S_1$ .  
Выигрыш на этом шаге:  $W_1(S_0, x_1)$ .
- **2-й шаг:** под действием управления  $x_2$  система переходит из состояния  $S_1$  в состояние  $S_2$ .  
Выигрыш:  $W_2(S_1, x_2)$ .
- ... и так далее.
- **$n$ -й шаг:** под действием управления  $x_n$  система переходит из состояния  $S_{n-1}$  в состояние  $S_n$ .  
Выигрыш:  $W_n(S_{n-1}, x_n)$ .

Здесь:

- $x_i$  — управление (переменная управления) на  $i$ -м шаге;
- $S_i$  — состояние системы после  $i$ -го шага;
- $W_i$  — целевая функция (выигрыш) на  $i$ -м шаге, называется **функцией Беллмана**.

Задача сводится к подбору такого управления  $X = (x_1, x_2, \dots, x_n)$ , которое переводит систему за  $n$  шагов из начального состояния  $S_0$  в конечное состояние  $S_n$  и обеспечивает **максимум** или **минимум** целевой функции в зависимости от условия задачи.

---

## 4. Функциональные уравнения Беллмана

Математическая формулировка принципа оптимальности представляется в виде функциональных уравнений.

Для последнего  $n$ -го шага:

$$W_n(S_{n-1}, x_n)$$

Для предпоследнего  $(n - 1)$ -го шага:

$$W_{n-1}(S_{n-2}, x_{n-1})$$

Аналогично для шага  $n - 2$  подбирается управление  $x_{n-2}$  и т.д.

---

## 5. Общая схема решения задачи динамического программирования

Процесс оптимизации управления методом динамического программирования — многошаговый, и он проходит **дважды**:

1. **Первый проход — от конца к началу.** Находится **условно оптимальное управление** на каждом шаге и выигрыш на всех шагах. Это процедура **условной оптимизации**.
  2. **Второй проход — от начала к концу.** Находится **безусловное оптимальное управление** на всех шагах процесса.
- 

## 6. Постановка задачи в общем виде

Пусть эффективность операции характеризуется показателем  $W$  — **выигрыш** (показатель эффективности операции).

Предположим, что выигрыш за всю операцию складывается из выигрышей на отдельных шагах:

$$W = \sum_{i=1}^m W_i$$

где  $m$  — число шагов,  $W_i$  — выигрыш на  $i$ -м шаге.

Операция представляет собой **управляемый процесс**: можно выбирать параметры, влияющие на ход и исход операции. На каждом шаге выбирается некоторое решение, независимое от выигрыша.

- $x$  — управление операцией в целом;
- $x_1, x_2, \dots, x_m$  — шаговые управления.

Требуется найти такое управление  $x$ , при котором выигрыш  $W$  будет **максимальным**.

Управление  $x^*$ , при котором достигается максимум выигрыша  $W$ , называется **оптимальным управлением**:

$$x^* = (x_1^*, x_2^*, \dots, x_m^*)$$

---

## 7. Свойства задач, решаемых методом динамического программирования

1. Задача должна допускать интерпретацию как  $n$ -шаговый процесс принятия решения (разбиение на этапы/шаги).
  2. Задача должна быть определена для любого числа шагов и иметь структуру, не зависящую от их числа.
  3. При рассмотрении каждого шага должно быть задано некоторое множество параметров, от которых зависит оптимальное значение.
  4. Выбор решения на  $k$ -м шаге не должен оказывать влияния на предыдущие решения (кроме пересчёта переменных).
- 

## 8. Классические и практические задачи

Задачи делятся на два типа:

- **Классические** — например, **задача о рюкзаке**.
- **Практические** — рассматриваются на занятиях:
  - задача распределения средств между предприятиями;
  - задача замены оборудования;
  - задачи распределения ресурсов.

**Задача о рюкзаке** (классическая): есть рюкзак заданной вместимости, имеется  $N$  предметов с определённым весом и ценностью. Требуется разместить предметы в рюкзаке так, чтобы получить максимальную ценность при минимальном возможном весе.

---

## 9. Пример: поиск наивыгоднейшего пути между двумя пунктами

Требуется проложить путь между пунктами  $A$  и  $B$ , причём  $B$  лежит к северо-востоку от  $A$ . Двигаться можно только строго на восток или строго на север (по диагонали — нельзя).

Любой путь из  $A$  в  $B$  — ступенчатая ломаная линия, отрезки параллельны осям  $X$  или  $Y$ . На каждом отрезке задана стоимость (затраты). Требуется найти путь с **минимальными суммарными затратами**.

Подходы:

1. **Перебор** всех возможных вариантов — долго и сложно.
2. **Динамическое программирование** — разбиение процесса перехода на отдельные шаги.

Исходные данные задачи:

- Расстояние от  $A$  до  $B$  в восточном направлении делится на **7 частей**, в северном — на **5 частей**.
- Любой путь состоит из **12 отрезков** (восток или север).
- На каждом отрезке задано число — стоимость перехода.

Алгоритм решения:

1. **Условная оптимизация** — движение от конца к началу (из  $B$  в  $A$ ):
  - Для каждой точки (кроме конечной) определяется управление (направление движения) и минимальная стоимость до конца пути.
  - Стоимость записывается в «кругляшок» рядом с точкой.

- Для точек, откуда есть только одно направление, управление — **вынужденное**.
- Если направлений два, выбирается направление с минимальной суммой: стоимость перехода + стоимость до конца.

Например, для последнего шага:

- точка  $B_1$ : идём на восток, расход 10;
- точка  $B_2$ : идём на север, расход 14.

Для предпоследнего шага (точки  $C_1, C_2, C_3$ ):

- $C_1$ : вынужденное управление на восток, итого  $11 + 10 = 21$ ;
- $C_2$ : выбирается между востоком ( $13 + 10 = 23$ ) и севером ( $13 + 14 = 27$ ), выбираем минимум — 23;
- $C_3$ :  $14 + 8 = 22$ .

2. В результате условной оптимизации в точке  $A$  получаем **условно оптимальный выигрыш**  $W^* = 118$  (записывается со звёздочкой).

3. **Безусловная оптимизация** — движение от начала к концу (из  $A$  в  $B$ ):

- Смотрим на стрелочки, полученные при условной оптимизации, и строим оптимальную траекторию.
- Оптимальная траектория отмечается жирными кружочками или стрелками.

4. Для сравнения: задача решается также «наивным» способом — на каждом шаге выбирается самое выгодное направление «здесь и сейчас», без учёта последствий. Для такого пути суммарные затраты равны  $W = 128$ , что **больше**  $W^* = 118$ . Это демонстрирует преимущество подхода динамического программирования.

Методические рекомендации:

- При условной оптимизации выделять выигрыши в кружочках (жирным или цветом).
- При «наивном» решении можно показывать стрелочки карандашом или другим цветом, чтобы различать траектории.

## 10. Выбор числа шагов

При постановке задач динамического программирования необходимо выбирать систему координат и способ разбиения на шаги с учётом ограничений, накладываемых на управление.

Рекомендации по выбору числа шагов:

1. Шаг должен быть **достаточно мелким**, чтобы процедура шаговой оптимизации была достаточно простой.
2. Шаг должен быть **не слишком маленьким**, чтобы не производить лишние мелкие расчёты.

Увеличение числа шагов ведёт к возрастанию объёма расчётов, что не всегда оправдано.

## 11. Задача о замене оборудования

### Постановка задачи

Вводятся обозначения:

- $r(t)$  — стоимость продукции, произведённой за один год на оборудовании возраста  $t$  лет;
- $u(t)$  — ежегодные затраты на обслуживание оборудования возраста  $t$  лет;
- $c(t)$  — остаточная стоимость оборудования возраста  $t$  лет;
- $p$  — покупная цена оборудования.

Рассматривается некоторый период времени в  $N$  лет. Требуется найти **оптимальный цикл замены оборудования**, то есть определить момент, когда оборудование выгоднее заменить, а не продолжать эксплуатировать.

Через  $f_N(t)$  обозначается максимальный доход от оборудования возраста  $t$  лет за оставшиеся  $N$  лет цикла.

Важная особенность: возраст оборудования отсчитывается в направлении течения процесса ( $t = 0$  — новое оборудование), а стадии процесса нумеруются в обратном направлении.

### Функциональные уравнения

На каждом этапе  $N$ -стадийного процесса принимается одно из двух решений:

- **сохранение** оборудования;
- **замена** оборудования.

Уравнения основаны на принципе оптимальности. Для одно- и многостадийного процесса используются функциональные уравнения Беллмана.

Для последнего шага, когда возраст оборудования равен  $N$  лет, некоторые слагаемые теряют смысл и исключаются из уравнений, поэтому уравнение записывается в упрощённом виде.

### Пример

Даны исходные данные:  $p$  — покупная цена,  $c(t)$  — остаточная стоимость,  $f(t) = r(t) - u(t)$  — чистый доход за год (стоимость произведённой продукции минус затраты на обслуживание). Значения заданы таблицей.

Используя функциональные уравнения, описываем процесс использования оборудования для каждой стадии. Вычисления продолжаются до тех пор, пока не выполняется условие замены:

$$f_{n-1}(1) > f_n(t)$$

В этот момент оборудование необходимо заменить.

Все результаты расчётов сводятся в таблицу. Для разобранный примера момент замены — **4 года**. Таким образом, оптимальный цикл замены оборудования составляет **4 года**: для получения максимальной прибыли в 12-этапном процессе оборудование следует заменять каждые 4 года.

---

## 12. Задача оптимального распределения ресурсов

### Постановка задачи

Имеется некоторое количество ресурсов  $x$ , которое необходимо распределить по  $n$  различным предприятиям (объектам) с целью получения максимальной суммарной эффективности.

Обозначения:

- $x$  — количество ресурсов;
- $g(x)$  — функция полезности;
- $f(x)$  — доход.

Математическая модель задачи:

$$F_n(x) = \max \sum_{i=1}^n f_i(x_i)$$

при ограничениях:

$$\sum_{i=1}^n x_i = x, \quad x_i \geq 0$$

### Рекуррентное соотношение

Для решения задачи необходимо получить рекуррентное соотношение, связывающее наибольший доход от первых  $k$  предприятий  $F_k(x)$  с  $F_{k-1}(x)$ .

Обозначим через  $x_k$  количество ресурсов, выделяемое  $k$ -му предприятию. Для максимизации суммарного дохода от  $k$  и  $k - 1$  первых способов используется соотношение:

$$F_k(x) = \max_{0 \leq x_k \leq x} [f_k(x_k) + F_{k-1}(x - x_k)]$$

### Пример

Совет директоров рассматривает предложение по наращиванию производственных мощностей для увеличения выпуска однородной продукции на **четырёх предприятиях**.

- Объём выделяемых средств: **120 млн рублей**.
- Дискретность выделения: **20 млн рублей** (т.е. средства выделяются порциями по 20 млн).
- Прирост выпуска продукции на каждом предприятии задан таблицей.
- Дополнительное условие: на одно предприятие можно осуществить **не более одной инвестиции**.

Решение разбивается на **4 этапа** (по количеству предприятий). Используются рекуррентные соотношения для каждого этапа:

1. **1-й этап** — инвестиции выделяются первому предприятию.
2. **2-й этап** — инвестиции распределяются между первым и вторым предприятиями.
3. **3-й этап** — между вторым и третьим.
4. **4-й этап** — между третьим и четвёртым.

После расчётов получается ответ: средства в объёме 120 млн рублей целесообразно выделить **второму, третьему и четвёртому предприятиям по 40 млн рублей каждому**. При этом максимальный прирост продукции составит **64** единицы.

Полученный ответ необходимо **интерпретировать**.

### 13. Достоинства и недостатки динамического программирования

#### Достоинства

- 1. Методы динамического программирования наиболее приспособлены к **дискретным задачам**.
- 2. Метод применим при любом способе задания целевой функции и любом допустимом множестве состояний и управлений (этого нет у классических методов оптимизации).
- 3. В дискретном случае вычислительные процедуры связаны с перебором оптимальных значений показателей эффективности и управления на каждом шаге.
- 4. Метод даёт возможность **анализа чувствительности** к изменению исходных данных.

#### Недостатки

- 1. Нет универсального алгоритма, пригодного для решения всех задач.
- 2. При большой размерности исходной задачи алгоритмы требуют больше времени и значительных вычислительных ресурсов.

### 14. Рекурсия и динамическое программирование

Рекурсию и динамическое программирование часто путают. Несмотря на сходство, их можно различить по четырём характеристикам: подход, определение, производительность, использование памяти.

| Характеристика     | Рекурсия                                                                                                   | Динамическое программирование                                                                              |
|--------------------|------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Определение        | Разбивает задачу на более мелкие подзадачи; функция вызывает саму себя до выполнения определённого условия | Разбивает задачу на подзадачи и <b>сохраняет результаты подзадач</b> , чтобы избежать повторных вычислений |
| Подход             | Сверху вниз                                                                                                | Снизу вверх                                                                                                |
| Производительность | Медленнее из-за избыточных повторных вычислений и вызовов функций                                          | Быстрее, так как оптимизирует подзадачи и запоминает их результаты                                         |
| Память             | Требуется больше памяти (стек вызовов)                                                                     | Требуется дополнительная память для хранения промежуточных результатов                                     |

### 15. «Жадные» алгоритмы

**Жадные алгоритмы** — алгоритмы, которые заключаются в принятии локально оптимальных решений на каждом этапе, допуская, что конечное решение также будет оптимальным.

Известные жадные алгоритмы:

- поиск кратчайшего пути — **алгоритм Флойда, алгоритм Дейкстры**;
- задача о минимальном покрывающем дереве;
- сжатие информации.

### Алгоритм Дейкстры (пример жадного подхода)

Используется для нахождения кратчайшего пути в графе с **неотрицательными весами рёбер**.

Начало работы:

- одной вершине (начальной) присваивается метка 0;
- остальным вершинам — очень большое число (или «бесконечность»).

Далее выполняется обход:

1. Выбирается необработанная вершина с наименьшей меткой.
2. Вычисляется расстояние до каждого соседа как сумма метки обрабатываемой вершины и веса ребра до соседа.
3. Если эта сумма меньше текущей метки соседа, метка обновляется.
4. После прохода по всем соседям вершина помечается как обработанная (зачёркивается).
5. Выбирается следующая необработанная вершина с наименьшей меткой, и процесс повторяется, пока все вершины не будут обработаны.

В лекции разобран пример на графе из 5 вершин: после обработки всех вершин получены кратчайшие расстояния от начальной вершины до остальных (например, метка  $V_4 = 17$ ).

### Преимущества жадных алгоритмов

1. **Простота** — легко реализуются и понимаются; логика интуитивно понятна.
2. **Эффективность** — работают быстрее других алгоритмов, особенно для задач, где жадный подход приводит к оптимальному решению.
3. **Меньше памяти** — не нужно хранить всевозможные решения.
4. Применимы в системах с ограниченными ресурсами.

### Недостатки жадных алгоритмов

1. **Не всегда оптимальны** — не дают 100% гарантии нахождения глобального оптимального решения для всех задач.
2. **Зависят от структуры задачи** — успешность применения сильно зависит от конкретной задачи и её подзадач.
3. **Ограниченность применения** — подходят не для всех типов задач; в некоторых случаях более сложные алгоритмы (динамическое программирование, метод ветвей и границ) могут быть эффективнее.

### Применение жадных алгоритмов

- **Маршрутизация** — оптимизация маршрутов доставки товаров, поиск кратчайшего пути.
  - **Задачи планирования** — распределение задач и ресурсов, оптимизация расписания.
  - **Сжатие данных** — алгоритмы Хаффмана для создания кодов сжатия, минимизация длины кода.
-

## Выводы по лекции

- Динамическое программирование — метод решения многошаговых задач оптимизации, основанный на принципе оптимальности Беллмана.
  - Процесс решения проходит дважды: сначала от конца к началу (условная оптимизация), затем от начала к концу (безусловная оптимизация).
  - Метод применим к дискретным задачам, позволяет анализировать чувствительность к исходным данным, но не имеет универсального алгоритма.
  - Рассмотрены три практические задачи: поиск кратчайшего пути, замена оборудования, распределение ресурсов.
  - Динамическое программирование отличается от рекурсии сохранением промежуточных результатов.
  - «Жадные» алгоритмы просты и быстры, но не всегда дают глобально оптимальное решение.
- 

## Вопросы для самопроверки (к зачёту)

1. Что такое динамическое программирование? Кем и когда оно было впервые предложено?
2. Сформулируйте принцип оптимальности Беллмана.
3. Что такое многошаговый процесс? Приведите примеры многошаговых операций.
4. Что называется функцией Беллмана?
5. Запишите функциональные уравнения Беллмана.
6. Опишите общую схему решения задачи динамического программирования (два прохода).
7. Какими свойствами должны обладать задачи для применения динамического программирования?
8. Какие рекомендации существуют по выбору числа шагов?
9. Как решается задача о замене оборудования методом динамического программирования?
10. Как решается задача оптимального распределения ресурсов?
11. Назовите достоинства и недостатки динамического программирования.
12. Чем рекурсия отличается от динамического программирования?
13. Что такое «жадные» алгоритмы? Приведите примеры. Каковы их преимущества и недостатки?